

Reviewer Pack — Cycle-Type Mixing Defect of Barrington BP Bounds NC^1 Length

Entry 816a64f0e4e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 03:58:46 UTC

Cycle-Type Mixing Defect of Barrington BP Bounds NC^1 Length

Entry ID: 816a64f0e4e2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 03:58:46 UTC

1. Conjecture

Field A (mathematical branch): Character-theoretic mixing on S_5 — the L^1 distance between the empirical cycle-type distribution of partial products of a deterministic word and the Haar cycle-type distribution $\pi^* = (1,10,15,20,20,24,30)/120$, in the Diaconis-Shahshahani 1981 / Lulov-Pak 2002 / Larsen-Shalev 2008 Plancherel-spectrum tradition. This is a STRUCTURAL invariant of the program (a property of the BP wiring, not of its truth table), and an arXiv search ‘conjugacy class mixing’ AND (‘branching program’ OR ‘Barrington’ OR ‘formula size’) returns 0 direct hits with <5 adjacent papers, all on quantum-circuit Haar approximation. **Field B** (complexity object): NC^1 formula leaf size via Barrington 1989 width-5 permutation branching programs over S_5 : every depth- d Boolean formula compiles into a length- 4^d width-5 BP over S_5 whose terminal permutation is the identity iff $f(x)=0$ and the fixed 5-cycle σ_α iff $f(x)=1$. The target separation is NC^1 vs explicit P functions; the structural form of the invariant shields it from the Razborov-Rudich natural-proofs barrier (two formulas with identical truth tables can compile to BPs with very different MIX).

Statement:

Let $P=(I_1, \dots, I_L)$ be a width-5 BP over S_5 computing a Boolean f via Barrington’s protocol, with $I_t=(v_t, \pi_t^{0, \pi_t^1})$. For uniform $X \in \{0,1\}^n$ let μ_t^P be the distribution on the 7 conjugacy classes of S_5 induced by $\Pi_{\{s \leq t\}} \pi_s^{X_{v_s}}$, and define $MIX(P) := (1/L) \cdot \sum_{t=1}^L \|\mu_t^P - \pi^*\|_1$. Conjecture: for every non-constant f and every width-5 Barrington BP P computing f via the standard Ben-Or-Cleve commutator gadget, $MIX(P) \cdot L \geq (1/8) \cdot \log_2 \text{LeafSize}(f)$, where $\text{LeafSize}(f)$ is the minimum literal-leaf count of any AND/OR/NOT formula computing f . A single (P, f) with $MIX(P) \cdot L < (1/8) \cdot \log_2 \text{LeafSize}(f)$ refutes the conjecture.

Rationale (proposer’s reasoning):

Barrington-compiled BPs realize commutators of 5-cycles whose conjugacy-class trajectory is forced to spend most steps in non-trivial classes when the underlying formula has high algebraic depth; π^* assigns 119/120 mass off the identity class, so the deterministic word can only stay ‘near uniform in cycle type’ by genuinely commutator-rich computation. Linking $MIX \cdot L$ to $\log \text{LeafSize}$ makes mixing a syntactic complexity certificate that is bypassed by no truth-table proxy, addressing the ACC_SIPSER target of finding natural properties that are STRUCTURAL (not truth-table). The bridge to Lulov-Pak character

bounds promises a path to $\text{ACC}^0[m]$ via Barrington-Thérien compilation into solvable iterated wreath products without invoking arithmetization (avoiding ALGEBRIZATION).

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f9ad9a91f4a3cc87

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds (formulas F with $d \in \{2, 3, 4\}$, $n \in \{6, 8, 10\}$) compiled to Barrington width-5 BPs P , SUPPORTED iff every instance satisfies $\text{MIX}(P) \cdot L \geq (1/8) \cdot \log_2 \text{LeafSize}(F)$ AND Spearman ρ between $\text{MIX} \cdot L$ and $\log_2 \text{LeafSize}$ across the 30 seeds is ≥ 0.5 . Any single violating (F, P) FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Barrington branching program S_5 conjugacy class mixing formula lower bound - width-5 permutation branching program character Plancherel $NC1$ formula size - Ben-Or Cleve commutator S_5 cycle-type distribution branching program formula leaf size

Top relevant hits considered: - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/1107.1826v4>] Conjugacy growth of finitely generated groups - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1512.02171v5>] Right-jumps and pattern avoiding permutations - [<http://arxiv.org/abs/1301.0379v2>] The Parameterized Complexity of some Permutation Group Problems - [<http://arxiv.org/abs/1301.1425v2>] Pebbling, Entropy and Branching Program Size Lower Bounds - [<http://arxiv.org/abs/1606.02278v1>] Perfect Commuting-Operator Strategies for Linear System Games - [<http://arxiv.org/abs/2408.11188v3>] Local-global principle for leaf schemes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(depth, n):
        if depth == 1:
            return f"x{random.randint(0, n-1)}"
        else:
            op = random.choice(['AND', 'OR'])
            a = generate_formula(depth - 1, n)
            b = generate_formula(depth - 1, n)
            return f"({a} {op} {b})"

    def leaf_size(formula):
        stack = []
        for char in formula:
            if char == '(':
                stack.append(char)
            elif char == ')':
                stack.pop()
        return len(stack) + 1

    def barrington_protocol(formula, n):
        depth = formula.count('AND') + formula.count('OR')
        L = 4 ** depth
        mu_t_P = [0] * 7
        for _ in range(L):
            input_bits = [random.randint(0, 1) for _ in range(n)]
            product = 1
            for bit in input_bits:
                if formula[bit] == 'x':
                    continue
                elif formula[bit].startswith('x'):
                    product *= int(formula[bit][1:])
                else:
                    product *= barrington_protocol(formula[bit], n)
            cycle_type = classify_cycle_type(product)
            mu_t_P[cycle_type] += 1
        for i in range(7):
            mu_t_P[i] /= L
        return sum(abs(mu_t_P[i] - pi_star[i]) for i in range(7))

    def classify_cycle_type(product):
        cycle_types = {
            1: [0, 1, 2], # Identity, (1), (2)
            2: [3, 4],    # (3), (4)
            3: [5],       # (5)
            4: [6],       # (6)
        }
        for key, values in cycle_types.items():
            if product in values:
                return key
        return -1

    pi_star = [Fraction(119, 120), Fraction(1, 120)] + [0] * 5

```

```

n_values = [6, 8, 10]
d_values = [2, 3, 4]
instances_tested = 0
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for d in d_values:
        formula = generate_formula(d, n)
        leaf_size_f = leaf_size(formula)
        L = 4 ** d
        metric_value = barrington_protocol(formula, n)
        instances_tested += L
        total_metric_value += metric_value * L
        if metric_value < (1/8) * math.log2(leaf_size_f):
            conjecture_holds = False
            counterexample = f"Formula: {formula}, LeafSize: {leaf_size_f}, MIX(P)*L: {metric_value}"

mean_metric_value = total_metric_value / instances_tested

return {
    "metric_name": "MIX(P) * L",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) # Default

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] * r["instances_tested"] for r in results) / sum(r["instances_tested"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c02f54b6.py", line 108, in <module>
    result = run_trial(seed)
            ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c02f54b6.py", line 85, in run_trial
    metric_value = barrington_protocol(formula, n)
                  ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c02f54b6.py", line 52, in barrington_protocol
    product *= barrington_protocol(formula[bit], n)
            ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c02f54b6.py", line 47, in barrington_protocol
    if formula[bit] == 'x':
        ~~~~~^~~~~~
```

IndexError: string index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in barrington_protocol before producing any RESULT line or data, so neither the support nor falsification condition could be evaluated. | next: Fix the recursive formula parser (handle empty subformulas and properly tokenize literals/operators) and re-run the 30-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	221278
2	preregistration	claude_max	opus	0	0	6226
3	novelty	claude_max	opus	0	0	4672
4	novelty	claude_max	opus	0	0	10197
5	test_gen	mistral	codestral-latest	0	0	320897
6	test_gen	mistral	codestral-latest	0	0	319074
7	test_gen	mistral	codestral-latest	0	0	313780
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	293744
9	judge	claude_max	opus	0	0	3602

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1493470 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/816a64f0e4e2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/816a64f0e4e2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/816a64f0e4e2.tar.gz` (if generated)